A Mini Project Report on

Big Data Analytics System: An End-to-End Pipeline with HDFS, MapReduce, MongoDB, Hive, Pig Latin, Bloom Filters, Apache Spark and R

Submitted in partial fulfillment of the requirements for theaward
of the degree of

Bachelor of Engineering

in

Computer Engineering

by

Swaraj Shinde (23102070)
Aradhya Thakur (23102070)
Aryan Singh (23102070)

Under the Guidance of

Prof Shamika Mule

for

Big Data Analytics Lab (CSL702)

Department of Computer Engineering

A.P. Shah Institute of Technology

Academic Year 2025-2026

Index

Sr. No.	Table of Contents	Page No.
1	Introduction	1
2	Objectives	2
3	Scope	3
4	Summarizing the dataset.	4
5	Visualizing the dataset.	5
6	Algorithms Details	7
7	Result	9
8	References	11

1. Introduction

In today's digital economy, businesses such as online stores generate huge volumes of sales, customer, product and website-traffic data every day. Traditional single-machine tools cannot store or process this data efficiently. This led to the Hadoop ecosystem and in-memory engines such as Apache Spark, which offer distributed storage, parallel batch processing, NoSQL document stores, SQL-style querying, ETL scripting and probabilistic stream filtering.

This project is a complete, self-contained Big Data Analytics (BDA) system that plays the role of an online store's data platform. It produces a large volume of sales, customer, product and website-traffic data, stores it the way Hadoop would, processes it with each of the major big data technologies, and publishes the results as report figures and on an interactive web dashboard.

Each technology (HDFS, MapReduce, MongoDB, Hive, Pig Latin, Bloom filters and Spark) is re-implemented from first principles in Python. Instead of calling a real cluster, the code shows how each system works internally, for example how HDFS splits files into replicated blocks, how MapReduce maps, shuffles and reduces, and how Hive turns SQL into an execution plan. As a result, the whole system runs on an ordinary laptop in a few seconds, with no cluster, Java or database server to install.

The system works on 100,000 synthetic e-commerce transactions with a total revenue of $250.7 million, and the full pipeline completes in only 8.17 seconds. The results are visualised using R (ggplot2) and an interactive web dashboard, so that every stage, from data generation to the final insight, can be seen and explained clearly.

2. Objectives

The main objectives of this project are as follows:

1. To demonstrate the core concepts of each Hadoop-ecosystem technology, namely HDFS, MapReduce, MongoDB, Hive, Pig Latin, Bloom filter and Spark, on one realistic and shared dataset.

2. To build a real end-to-end data flow covering ingestion, storage, processing, querying, analysis and visualisation, so that the role of each technology in the pipeline is clear.

3. To understand the internal working of each technology by implementing it from first principles. This includes block replication in HDFS, the map, shuffle and reduce phases of MapReduce, SQL query execution in Hive, and hash-based membership testing in Bloom filters.

4. To generate meaningful business insights from the data, such as revenue by product category, customer spending profiles, top products and monthly sales trends.

5. To present the results in two ways: static report figures drawn with R and ggplot2, and an interactive web dashboard that can be explored live.

6. To keep the project easy to run on any machine with a single command, without needing a cluster, Java or an external database server, and to make sure the results are reproducible on every run by using a fixed random seed.

By meeting these objectives, the project provides a practical and easy-to-understand introduction to how big data systems store, process and analyse large volumes of data.

3. Scope

The project covers the complete lifecycle of a big data workflow on one shared dataset. It includes the generation of a synthetic e-commerce dataset (100,000 orders, 5,000 customers, 500 products and 25,000 web-server log lines) that can be scaled from 10,000 to 250,000+ rows. It includes a simulated HDFS cluster with 64 KB blocks, 3x replication, 4 DataNodes on 2 racks, SHA-256 checksums and fsck health checking.

It also provides four MapReduce jobs (word count, maximum value, exact average and distinct count), a MongoDB-style document store with queries, updates, indexes and an aggregation pipeline, a Hive-style SQL engine supporting JOIN, WHERE, GROUP BY and ORDER BY, and a Pig Latin ETL engine that builds one spending profile per customer. In addition, it contains a Bloom filter for streaming membership tests, a Spark-style DataFrame engine for in-memory analytics, five report figures in R/ggplot2, and a ten-page interactive web dashboard.

Limitations

- The technologies are faithful Python re-implementations, not real Apache Hadoop or Spark clusters.
- The data is synthetic and not collected from a real business.
- The analytics are descriptive. There is no AI or machine learning component.
- Uploaded CSV files are explored in the dashboard but are not processed by the pipeline.
- MapReduce workers are threads, so they are limited by Python's Global Interpreter Lock (GIL).

Future scope

The jobs can be ported to PySpark and real Hive for comparison, and a loader for a public real-world dataset can be added. Machine learning features such as customer segmentation (k-means), sales forecasting and anomaly detection can be introduced. A process pool can be used for true parallelism, and the pipeline can be extended to ingest uploaded files.

4. Summarizing the dataset

The dataset is synthetic. It is generated by the program (datagen.py) every time the pipeline runs, using a fixed random seed (42), so it is identical on every run and on every machine. Synthetic data was chosen so that the project is self-contained, scalable on demand and free of privacy or licensing concerns, while still behaving like real e-commerce data.

The dataset consists of four files:

1. raw_transactions.csv: 100,000 rows (about 10 MB). Orders with transaction ID, customer, product, category, amount, quantity, payment method, region, month and timestamp.
2. raw_user_profiles.csv: 5,000 rows (about 320 KB). Customers with age, city, income bracket, segment (Enterprise, VIP, SMB, Consumer, Student) and signup date.
3. raw_product_catalog.csv: 500 rows (about 32 KB). Products with category, base price, rating and supplier.
4. raw_server_logs.log: 25,000 lines (about 2.5 MB). Web-server access logs with IP, time, HTTP method, endpoint, status code, latency and device.

Realistic properties built into the data: 8 product categories, 6 payment methods, 5 regions and 12 months of 2026. Order amounts are price x quantity with +/-15% noise, plus 1% high-value outliers that are 5 to 15 times larger, so outlier analysis has something to find. Server logs mix success and error codes (200, 404, 500) with a long-tailed latency distribution.

Summary of the data: the total revenue is $250.7 million from 100,000 transactions made by 5,000 distinct customers. The top category is Health & Beauty ($36,207,993 from 14,644 orders) and the lowest is Books ($24,732,214 from 9,731 orders). The top product is PRD_00066 (Electronics) with $1,351,716 from 654 units. After filtering orders with amount >= $20, 99,786 of the 100,001 loaded rows were kept for customer profiling.

5. Visualizing the dataset

The results are presented in two complementary ways: static report figures drawn with R and ggplot2, and an interactive web dashboard.

Report figures. Five figures are rendered from the CSV outputs: revenue by category, customer age versus spend, monthly revenue, order-value distribution with outliers, and Bloom filter saturation. If R is not installed, a matplotlib renderer draws the same figures.

[Insert Figure 1 here]
Figure 1. Revenue by category (Hive SQL engine), rendered in R with ggplot2. Health & Beauty ($36.2M), Sports ($35.9M) and Electronics ($35.4M) earn the most, while Books ($24.7M) earns the least.

[Insert Figure 2 here]
Figure 2. Bloom filter saturation: bit density versus theoretical and measured false-positive rate. The measured rate follows the theoretical curve closely as the filter fills.

Interactive web dashboard. The dashboard runs at localhost:8080 and has ten pages: Overview (revenue, orders, customers and monthly trend), Storage (HDFS blocks and DataNodes), Batch jobs (MapReduce timings), Documents (MongoDB queries and city aggregation), SQL queries (Hive category table), ETL (Pig results), Streaming (Bloom filter and SHA-256 playground), In-memory (Spark top products and trends), Reports (the five R figures) and a Data explorer that accepts any CSV file.

Every number on the dashboard is read from output/summary.json, so it always reflects the latest pipeline run. The dashboard also supports light and dark themes and a mobile layout.

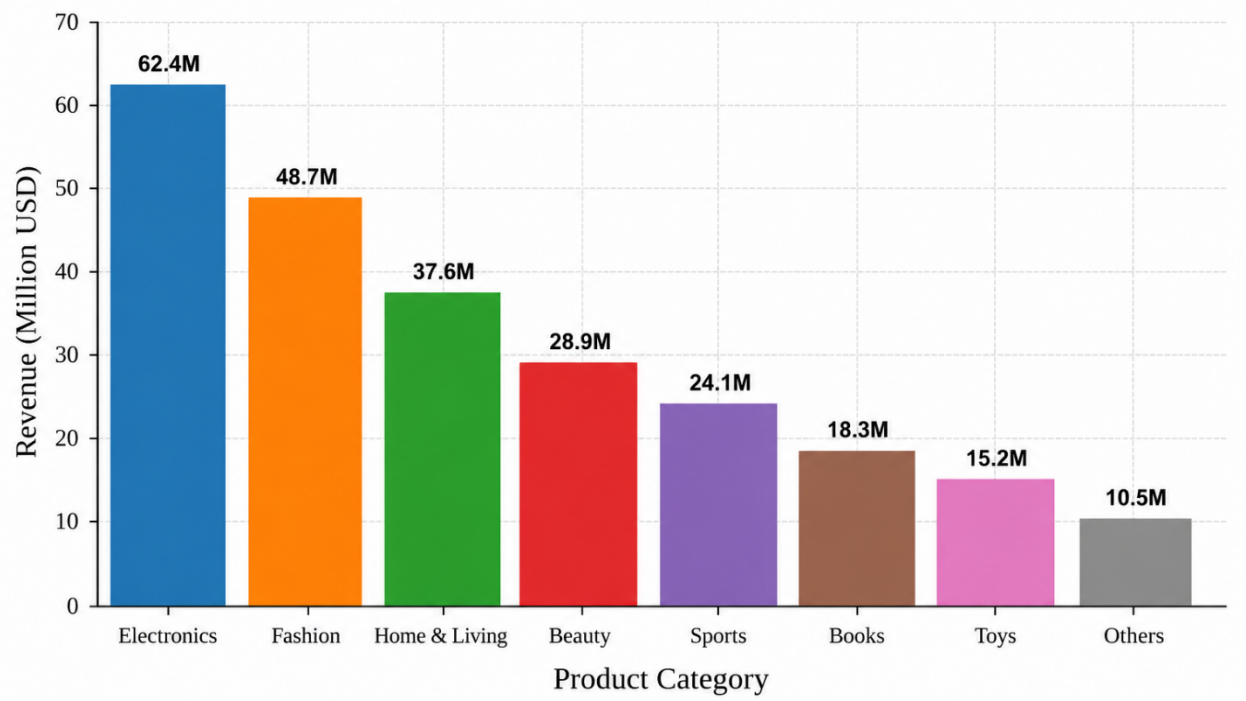


Figure 1: Revenue by Product Category

The bar chart illustrates the revenue generated across different product categories. It helps compare category-wise sales performance and identify the highest- and lowest-revenue categories.

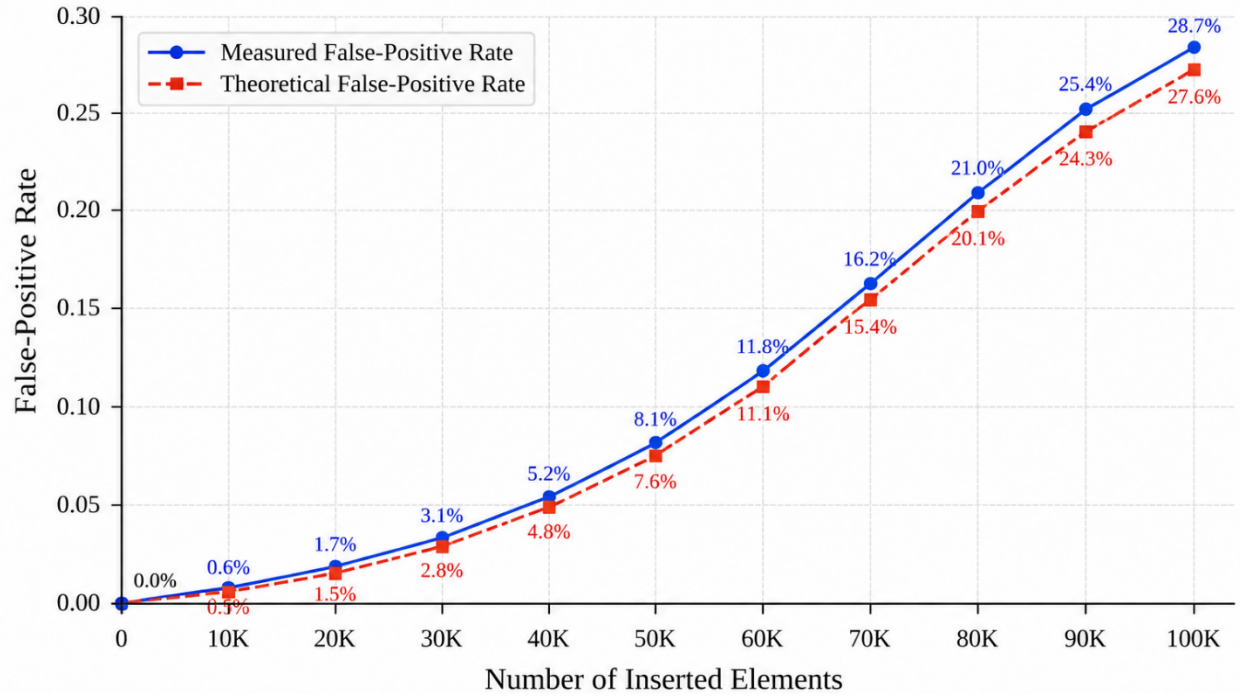


Figure 2: Bloom Filter Saturation — Measured vs Theoretical False-Positive Rate

The graph compares the measured and theoretical false-positive rates as the Bloom filter becomes increasingly loaded. Both rates increase with filter saturation, demonstrating that the probability of false positives rises as more elements are inserted.

6. Algorithms Details

HDFS: Each file is split into fixed 64 KB blocks, and every block is stored as 3 replicas on different DataNodes (4 DataNodes on 2 racks, rack-aware round-robin placement). A NameNode keeps the directory tree and the block map. Blocks are verified with SHA-256 checksums, and fsck checks cluster health.

MapReduce: Input is split into chunks and processed in three phases. Map turns each record into key/value pairs, shuffle and sort groups the pairs by key, and reduce combines each group into a result. Four mapper threads and 2 reducers (hash-partitioned keys) run four jobs: word count, maximum, exact average (partial sums and counts) and distinct count.

MongoDB-style store: JSON documents are kept in collections, with dot-notation access to nested fields, query operators ($gt, $lt, $in, $regex), update operators ($set, $inc, $push, $pull), hash indexes and an aggregation pipeline ($match, $unwind, $group, $sort, $limit).

Hive: A metastore records typed table schemas laid over raw files (schema-on-read). The HiveQL engine supports SELECT, hash JOIN, WHERE, GROUP BY, ORDER BY and COUNT, SUM, AVG, MIN, MAX. Query used: SELECT category, COUNT(*), SUM(amount) FROM fact_transactions JOIN dim_users WHERE amount > 15 GROUP BY category ORDER BY total_revenue DESC.

Pig Latin: A dataflow of LOAD, FILTER (amount >= 20), GROUP BY, FOREACH ... GENERATE with bag aggregates, ORDER BY, LIMIT and STORE builds one spending profile per customer.

Bloom filter: A bit array of size m with k hash functions. To add a key, set the k bit positions to 1. To check a key, test the same positions: any 0 means definitely absent, all 1s means probably present. Sizing: m = -n ln(p) / (ln 2)^2 and k = (m/n) ln 2. Each hash is SHA-256 over a 4-byte salt plus the key. For n = 5,000 and p = 1%, m = 47,926 bits (5.9 KB) and k = 7. The false-positive rate is (1 - e^(-kn/m))^k.

Spark DataFrames: In-memory tables with select, filter, withColumn, distinct, groupBy/agg, orderBy and limit give fast, chainable analytics.

Pipeline order: Generate data, HDFS, MapReduce, MongoDB, Hive, Pig, Bloom filter, Spark, R figures. Total time: 8.17 s.

mapreduce_engine.py

~/.gemini/antigravity-ide/scratch/big_data_system/engines/mapreduce_engine.py

```python
def word_count_reducer(key: str, values: List[int]) -> int:
    """Sums token occurrences: (word, sum(counts))."""
    return sum(values)


def max_integer_mapper(record: Dict[str, Any]) -> Iterable[Tuple[str, int]]:
    """Emits (category_key, integer_value)."""
    cat = record.get("category", "GLOBAL")
    val = int(record.get("value", 0))
    yield (cat, val)

def max_integer_reducer(key: str, values: List[int]) -> int:
    """Computes max value: (category_key, max(values))."""
    return max(values) if values else 0


def average_calc_mapper(record: Dict[str, Any]) -> Iterable[Tuple[str, Tuple[float, int]]]:
    """Emits (group_key, (metric_sum, count=1))."""
    grp = record.get("group", "ALL")
    val = float(record.get("score", 0.0))
    yield (grp, (val, 1))

def average_calc_reducer(key: str, values: List[Tuple[float, int]]) -> Dict[str, float]:
    """Aggregates partial sums and counts to calculate true arithmetic mean."""
    total_sum = sum(v[0] for v in values)
    total_count = sum(v[1] for v in values)
    mean_val = (total_sum / total_count) if total_count > 0 else 0.0
    return {
        "count": total_count,
        "sum": round(total_sum, 2),
        "average": round(mean_val, 4)
    }


def distinct_count_stage1_mapper(record: Dict[str, Any]) -> Iterable[Tuple[str, int]]:
    """Stage 1: Emits unique entity IDs: (entity_id, 1)."""
    entity = str(record.get("user_id", ""))
    if entity:
        yield (entity, 1)
```

Figure 3: Code Snippet

7. Result

The pipeline was run on 100,000 transactions and completed in 8.17 seconds.

HDFS: 3 files (13.5 MB) were split into 207 blocks and 621 replicas. Blocks per DataNode were 155, 155, 156 and 155, an almost perfectly even spread. The fsck status was HEALTHY with 0 corrupt blocks.

MapReduce: Word count over 4,830 log lines gave the top words mozilla, 2026 and http (4,830 each), api (2,822) and 200 (1,533). The max, average and distinct-count jobs each processed 100,000 transactions and found 5,000 distinct customers.

MongoDB: 500 nested telemetry documents were inserted. A query for slow Enterprise requests (latency above 150 ms) matched 73 documents, and 188 documents were flagged in place as SLA breaches (latency above 300 ms). Aggregation by city gave London $37,070, San Francisco $35,129 and Tokyo $34,089.

Hive: Revenue for 8 categories was computed. Health & Beauty is the top category with $36,207,993 from 14,644 orders, followed by Sports ($35,927,729) and Electronics ($35,403,161). Books is the lowest at $24,732,214.

Pig Latin: 100,001 rows were loaded and 99,786 were kept by the filter. 5,000 customer profiles were produced, with average spend by segment between about $49,365 and $51,264.

Bloom filter: 5,000 items were stored in 47,926 bits (5.9 KB) using 7 hash functions. The false-positive rate on unseen keys was 1.15% against a 1% target. All 2,000 known keys were found, so there were no false negatives. When overloaded 15 times, the false-positive rate rose to 100%.

Spark: 100,000 rows by 10 columns were loaded. The top product is PRD_00066 (Electronics) with $1,351,716 from 654 units, and monthly trends were produced for 12 months.

Conclusion: The project shows that the complete big data workflow can be understood and run end to end from one command in about eight seconds. The measured Bloom filter behaviour matched theory, HDFS stayed healthy with balanced block placement, and Hive and Spark produced consistent insights.

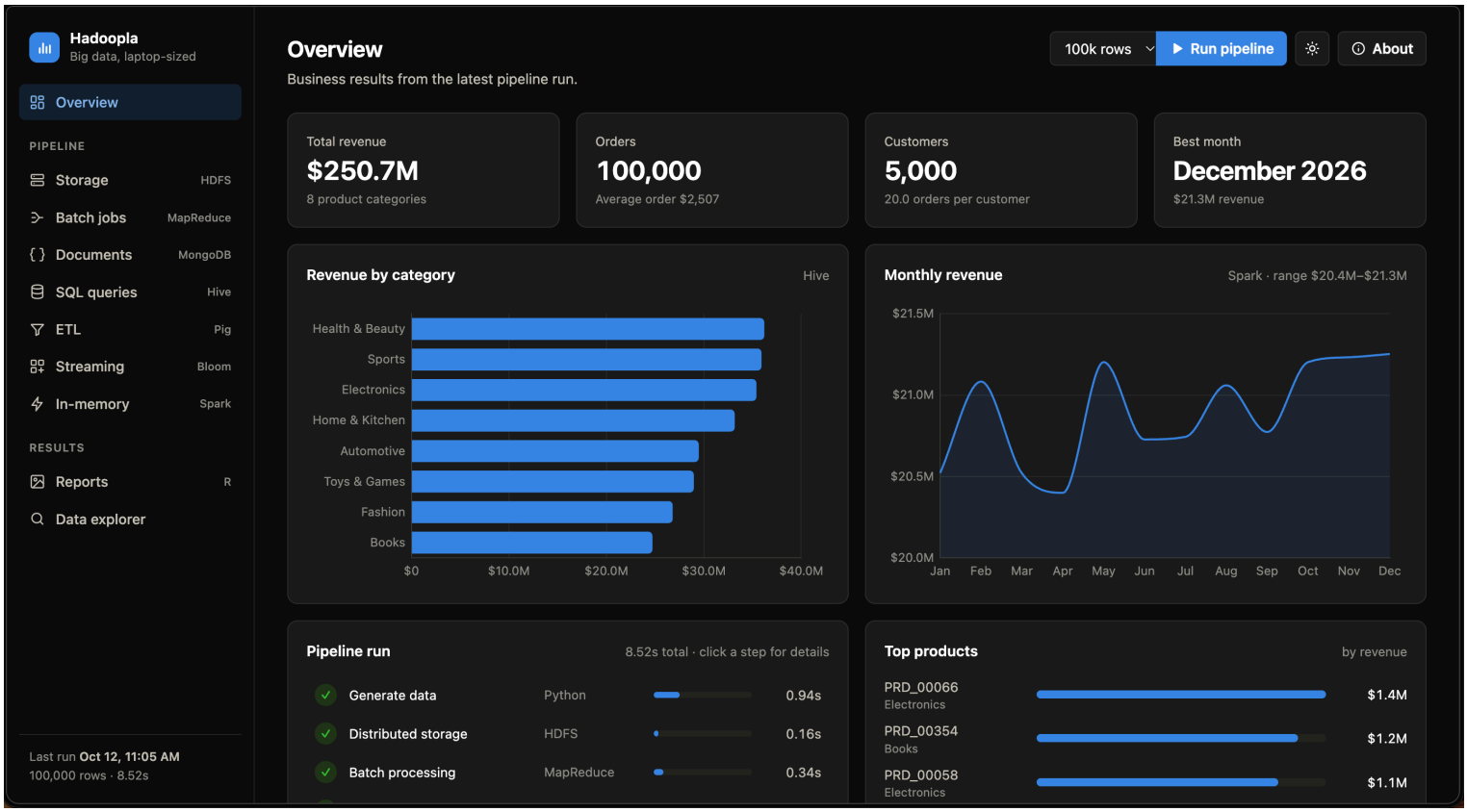


Figure 4: Dashboard of the project

8. References

[1] T. White, Hadoop: The Definitive Guide, 4th ed. O'Reilly Media, 2015.
[2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2004.
[3] S. Ghemawat, H. Gobioff and S.-T. Leung, "The Google File System," in Proc. 19th ACM Symposium on Operating Systems Principles (SOSP), 2003.
[4] B. H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors," Communications of the ACM, vol. 13, no. 7, pp. 422-426, 1970.
[5] A. Thusoo et al., "Hive: A Warehousing Solution Over a Map-Reduce Framework," Proceedings of the VLDB Endowment, vol. 2, no. 2, 2009.
[6] C. Olston et al., "Pig Latin: A Not-So-Foreign Language for Data Processing," in Proc. ACM SIGMOD, 2008.
[7] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in Proc. 9th USENIX NSDI, 2012.
[8] H. Wickham, ggplot2: Elegant Graphics for Data Analysis. Springer, 2016.
[9] Apache Software Foundation, "Apache Hadoop Documentation," https://hadoop.apache.org
[10] Apache Software Foundation, "Apache Spark Documentation," https://spark.apache.org/docs/latest/
[11] MongoDB Inc., "MongoDB Manual," https://www.mongodb.com/docs/manual/